BCSCHOOL

Empowering Digital Skills For The Jobs Of The Future

by

BC Soft
consulenza sistemi informativi

AiRED
Artificial Intelligence Ricerca e Sviluppo

JAVA JEE
-
Introduzione a Stile di codifica, tipi di dati ed array

TEACHER

Vincenza Ruvolo

vincenza.ruvolo@bcsoft.net

Sommario

- Stile di codifica
- Tipi di dati primitivi
- Tipi di dati non primitivi: reference
- Introduzione alla libreria standard
- Gli array in Java
- Laboratorio

Stile di codifica

Con il termine stile di codifica si intende una serie di regole obbligatorie e non, che caratterizzano solitamente il codice Java.

Stile di codifica

Il linguaggio Java:

- È a schema libero.
- Supporta vari tipi di commenti.
- Definisce parole chiave.
- È case sensitive.
- Ha regole ferree per i tipi di dati ed alcune semplici convenzioni per i nomi.

Schema Libero

Java è un linguaggio a schema libero.

Questo, però, non significa che si possono violare le regole del linguaggio.

Schema Libero

Le Regole del linguaggio devono seguire il seguente ordine:

- Dichiarazione dei package
- Istruzioni di importazione
- Dichiarazione di una classe

Schema Libero

Java è a schema libero nel senso che si potrebbe scrivere il codice tutto su un’unica riga o su righe diverse e il compilatore non darebbe errore.

Metodi di formattazione

Esistono ad ogni modo metodi standard di indentazione che rendono più leggibile il codice scritto.

Lo stile più utilizzato dai programmatori Java è il seguente:

Metodi di formattazione

```
public class Classe {

    public int intero;

    public void metodo() {
        intero = 29;
        int unAltroIntero = 7;
        System.out.println(intero + unAltroIntero);
    }
}
```

Commenti

Java supporta tre modi diversi per commentare il codice:

- **Primo metodo**

Commenti a singola riga (letteralmente single line comments):

```
// Questo è un commento su una sola riga
```

Commenti

- **Secondo metodo:**

Commenti multilinea (letteralmente multiline comments):

```
/*
 * Questo è un commento
 * su più righe
 */
```

Commenti

- **Terzo metodo**

Commenti di documentazione (letteralmente documentation comments), conosciuti anche come commenti Javadoc:

```
/**
* Questo commento permette di produrre
* la documentazione del codice
* in formato HTML, nello standard Javadoc
*/
```

Documentazione tecnica

La “documentazione tecnica” spiega il funzionamento dei componenti Java (es. metodi, costruttori) descrivendo i parametri di input, quelli di output, e il comportamento (ovvero ciò che fa) con una sintassi precisa e standard.

Regole per gli identificatori

Gli identificatori (sinonimo di nomi) dei metodi, delle classi, degli oggetti, dei package, delle variabili e delle costanti (ma anche delle interfacce, delle enumerazioni e delle annotazioni che studieremo più avanti) hanno due regole da rispettare obbligatoriamente.

Regole per gli identificatori

- **Prima regola:**

Java può usare solo un certo insieme di caratteri, numeri e simboli.

Regole per gli identificatori

In particolare, in un identificatore:

- il primo carattere può essere A-Z, a-z, _, $
- il secondo ed i successivi possono essere A-Z, a-z, _, $, 0-9

Esempio: a2 è un identificatore valido, mentre 2a non lo è.

Regole per gli identificatori

- **Seconda regola:**

Un identificatore non può coincidere con una parola chiave (keyword) di Java. Infatti, una parola chiave ha un certo significato per il linguaggio di programmazione.

Regole per gli identificatori

Le parole chiave in Java sono:

abstract	assert	boolean	break	byte	case
catch	char	class	const	continue	default
do	double	else	enum	exports	extends
final	finally	float	for	if	goto
implements	import	instanceof	int	interface	long
module	native	new	open	opens	package
private	protected	provides	public	requires	return
short	static	strictfp	super	switch	synchronized
this	throw	throws	to	transient	transitive
try	uses	void	volatile	while	with
@interface	_				

Regole per gli identificatori

Non tutte le parole presenti in tabella sono keyword.

Le seguenti parole, ad esempio, prendono il nome di **parole riservate** (reserved word):

goto, const e _

Regole per gli identificatori

Queste, infatti, non hanno un significato per il linguaggio, ma semplicemente non si possono utilizzare come identificatori.

Regole per gli identificatori

Le parole chiave **goto** e **const** sono parole riservate da quando è nato il linguaggio, mentre per quanto riguarda _ è una novità di Java 9.

Regole per gli identificatori

Le seguenti parole:

exports	opens	to	uses
module	provides	transitive	with
open	requires		

vengono dette **parole a utilizzo limitato** (restricted word).

Standard e convenzioni per i nomi

Gli identificatori devono essere significativi.

Di solito l'identificatore di una variabile è composto da uno o più sostantivi, per esempio *numeroLati* o *larghezza*.

Standard e convenzioni per i nomi

Esistono convenzioni per gli identificatori a seconda del tipo di componente a cui bisogna dare un nome, esse sono:

- Convenzioni per le classi
- Convenzioni per le variabili
- Convenzioni per i metodi

Standard e convenzioni per i nomi

- **Convenzione per le classi**

Deve sempre iniziare con una lettera maiuscola e se composto da più parole, queste non si possono separare, perché il compilatore non può intuire le nostre intenzioni.

Standard e convenzioni per i nomi

Esempio:

- Persona
- MacchinaDaCorsa
- FiguraGeometrica

Standard e convenzioni per i nomi

- **Convenzione per le variabili**

Un identificatore di una variabile deve sempre iniziare con una lettera minuscola.

Se l'identificatore di una variabile deve essere composto da più parole, valgono le stesse regole in uso per gli identificatori delle classi.

Standard e convenzioni per i nomi

Esempio:

- pesoSpecifico
- numeroDiMinutiComplessivi
- x

Standard e convenzioni per i nomi

- **Convenzione per i metodi**

Per un identificatore di un metodo valgono le stesse regole in uso per gli identificatori delle variabili.

È possibile distinguere un identificatore di una variabile da un identificatore di un metodo, poiché quest'ultimo è sempre seguito da parentesi tonde.

Standard e convenzioni per i nomi

Il nome di un metodo dovrebbe contenere almeno un verbo.

Esempio:

- sommaDueNumeri(int a, int b)
- cercaUnaParola(String parola)
- stampa()

Standard e convenzioni per i nomi

- **Convenzione per le costanti**

Si devono distinguere nettamente dagli altri, quindi, tutte le lettere dovranno essere maiuscole.

Se l’identificatore è composto di più parole, queste si separano con un underscore.

Standard e convenzioni per i nomi

Esempio:

- NUMERO_LATI_DI_UN_QUADRATO
- PI_GRECO

Standard e convenzioni per i nomi

Convenzione per i package

I nomi dei package contengono generalmente solo lettere minuscole.

Se occorre utilizzare parole composte è possibile creare package innestati.

Standard e convenzioni per i nomi

Esempio:

- Se si volesse creare un package di assistenza clienti, è possibile creare due package innestati nel seguente modo:

assistenza.clienti

Tipi di dato

Java prevede i seguenti tipi di dato:

1. Primitivi (o tipi base)

2. Riferimento

TIPI	**punta a** →	**VALORE**
• Classi e Interfacce		oggetto
• Array		(oggetto) array
• Tipo nullo		null

Tipi di dati primitivi

In Java sono definiti otto tipi di dati primitivi:

- **Tipi interi:** byte, short, int, long.
- **Tipi floating point** (o a virgola mobile): float e double.
- **Tipo testuale:** char.
- **Tipo logico-booleano:** boolean.

Tipi di dati primitivi

I tipi di dati interi condividono la stessa funzionalità:

tutti possono immagazzinare numeri interi positivi o negativi

Tipi di dati primitivi

I tipi di dati interi differiscono per quanto riguarda il proprio intervallo di rappresentazione, compreso tra un minimo di -2 elevato al numero di bit meno uno, ed un massimo di 2 elevato al numero di bit meno uno.

$$\left[-2^{n\ bit\ -1}, 2^{n\ bit-1} - 1\right]$$

Tipi di dati primitivi

Tipo	Intervallo di rappresentazione
`byte`	8 bit (da -2^7 a 2^7-1)
`short`	16 bit (da -2^{15} a $2^{15}-1$)
`int`	32 bit (da -2^{31} a $2^{31}-1$)
`long`	64 bit (da -2^{63} a $2^{63}-1$)

Tipi di dati primitivi

Per memorizzare i numeri decimali, Java utilizza i tipi a virgola mobile (in inglese floating point). Si parla di virgola mobile perché un numero decimale è possibile rappresentarlo spostando la virgola utilizzando la notazione scientifica.

Tipi di dati primitivi

Per esempio, 0,0001 è equivalente a $1 * 10^{-4}$.

Un primo esempio di dichiarazione di double potrebbe essere il seguente:

```
double d = 0.0126;
```

Tipi di dati primitivi

Il tipo di dato boolean utilizza solo un bit per memorizzare un valore e gli unici valori che può immagazzinare sono true e false.

Tipi di dati primitivi

- Tipo di default dei valori interi: int (tipo a 32 bit).
- Tipo di default dei valori floating point: tipo double (64 bit) e float (32 bit).

Tipi di dati primitivi

Nella libreria standard sono definite delle classi, dette classi Wrapper (in italiano classi involucro), che rappresentano i tipi di dati primitivi.

Tipi di dati primitivi

Primitive Type	Size	Minimum Value	Maximum Value	Wrapper Type
char	16-bit	Unicode 0	Unicode 2^{16}-1	Character
byte	8-bit	-128	+127	Byte
short	16-bit	-2^{15} *(-32,768)*	$+2^{15}$-1 *(32,767)*	Short
int	32-bit	-2^{31} *(-2,147,483,648)*	$+2^{31}$-1 *(2,147,483,647)*	Integer
long	64-bit	-2^{63} *(-9,223,372,036,854,775,808)*	$+2^{63}$-1 *(9,223,372,036,854,775,807)*	Long
float	32-bit	32-bit IEEE 754 floating-point numbers		Float
double	64-bit	64-bit IEEE 754 floating-point numbers		Double
boolean	1-bit	`true` or `false`		Boolean
void	-----	-----	-----	Void

Tipi di dati primitivi

Vengono definiti **literals** i valori contenenti lettere o simboli che vengono specificati nel codice sorgente invece che a runtime.

Tipi di dati primitivi

Esiste anche la possibilità di immagazzinare caratteri di escape.

Sequenza	Significato
`\b`	backspace
`\t`	tabulazione
`\n`	newline
`\r`	ritorno
`\"`	virgolette doppie
`\'`	virgoletta
`\\`	backslash

Tipi di dati primitivi

Il tipo **char** permette di immagazzinare caratteri (uno per volta).

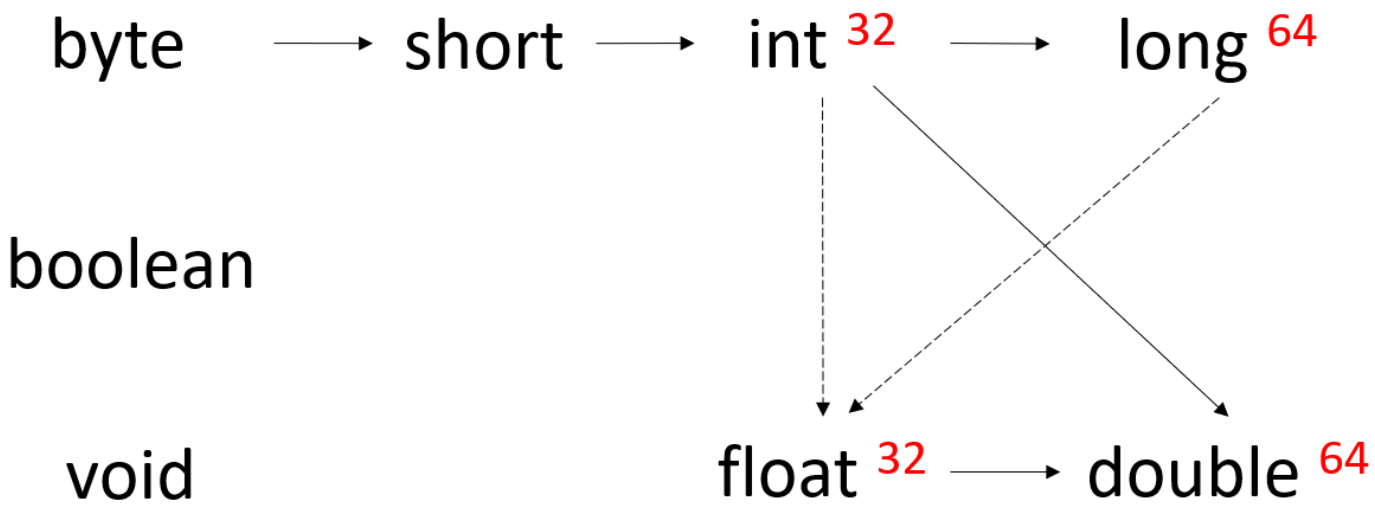
Questo tipo di dato viene memorizzato in 2 byte (16 bit) e utilizza lo standard di decodifica dei caratteri noto come Unicode.

Unicode

Unicode può essere utilizzato in tre modi:

1. **UTF-8:** a 8-bit che coincide con la codifica ASCII.
2. **UTF-16:** a 16 bit, che contiene tutti i caratteri della maggior parte degli alfabeti del mondo. Anche il char è a 16 bit e quindi con esso si è in grado di rappresentare praticamente tutti i caratteri più significativi esistenti.
3. **UTF-32:** a 32 bit che contiene codifiche di altri caratteri compresi quelli che potrebbero essere considerati meno usati.

Conversioni implicite



Conversioni implicite

Se c’è un percorso nel grafo allora vi è una conversione implicita.

Il percorso può avere sia linee continue che linee tratteggiate. Le linee tratteggiate indicano perdita di informazione.

boolean e void non sono confrontabili con nessuno.

Tipi di dati non primitivi: reference

Quando bisogna istanziare oggetti da una certa classe, bisogna prima dichiarare un oggetto di tale classe con una sintassi di questo tipo:

NomeClasse nomeOggetto;

per poi istanziarlo utilizzando la parola chiave new.

Tipi di dati non primitivi: reference

Dichiarare un oggetto è simile a dichiarare un tipo di dato primitivo.

Il “nome” che si dà ad un oggetto è detto **reference (o riferimento).**

Tipi di dati non primitivi: reference

Il tipo di dato riferimento, è spesso definito puntatore, infatti è possibile definire un puntatore come una variabile che contiene un indirizzo in memoria (il valore di un reference coincide con l’indirizzo in memoria).

Tipi di dati non primitivi: reference

Tutti i tipi non primitivi sono tipi reference.

Una variabile di tipo reference non è altro che un puntatore ad un oggetto.

Passaggio di parametri per valore

“Il passaggio di parametri in Java avviene sempre per valore”.

Quando si invoca un metodo che come parametro prende in input una variabile, al metodo stesso viene passato solo il valore (una copia) della variabile che quindi rimane immutata anche dopo l’esecuzione del metodo.

Passaggio di parametri per valore

Nel momento in cui si passa un valore primitivo ad un parametro di un metodo, verrà copiato nel parametro questo valore.

Quando si passa un reference, verrà passato il suo valore (l'indirizzo del reference) e copiato nel parametro del metodo.

Inizializzazione delle variabili d’istanza

Nel momento in cui è istanziato un oggetto, tutte le sue variabili d’istanza vengono inizializzate ai rispettivi valori nulli.

Inizializzazione delle variabili d’istanza

Di seguito è presentata una tabella che associa ad ogni tipo di dato il valore con cui viene inizializzata una variabile di istanza al momento della sua creazione:

Passaggio di parametri per valore

Variabile	Valore
byte	0
short	0
int	0
long	0L
float	0.0f
double	0.0d
char	'\u0000' (**Valore nullo - nessun carattere**)
boolean	false
Ogni tipo reference	null

Libreria standard

Java possiede un’enorme e lussuosa libreria di classi standard che costituisce uno dei punti di forza del linguaggio. È anche nota come Java API, dove API è l’acronimo per “Application Programming Interface”.

Libreria standard

I principali package sono:

- **java.io** contiene classi per realizzare l'input/output in Java.
- **java.awt** contiene classi per realizzare interfacce grafiche (es. Button).

Libreria standard

- **java.net** contiene classi per realizzare connessioni (es. Socket).
- **java.util** raccoglie classi d'utilità (es. Date).
- **java.lang** è il package che contiene le classi nucleo del linguaggio (es. System, String).

Libreria standard

Per utilizzare una classe della libreria all’interno di una classe che si ha intenzione di scrivere, bisogna prima importarla, utilizzando **import.**

Libreria standard

Si supponga di voler utilizzare la classe Date del package java.util, quindi bisogna scrivere:

import java.util.Date

La classe String

In Java, le stringhe sono oggetti, e in quanto tali dovrebbero essere istanziate con la solita sintassi tramite la parola chiave new.

Per esempio:

```
String nome = new String("Mario Rossi");
```

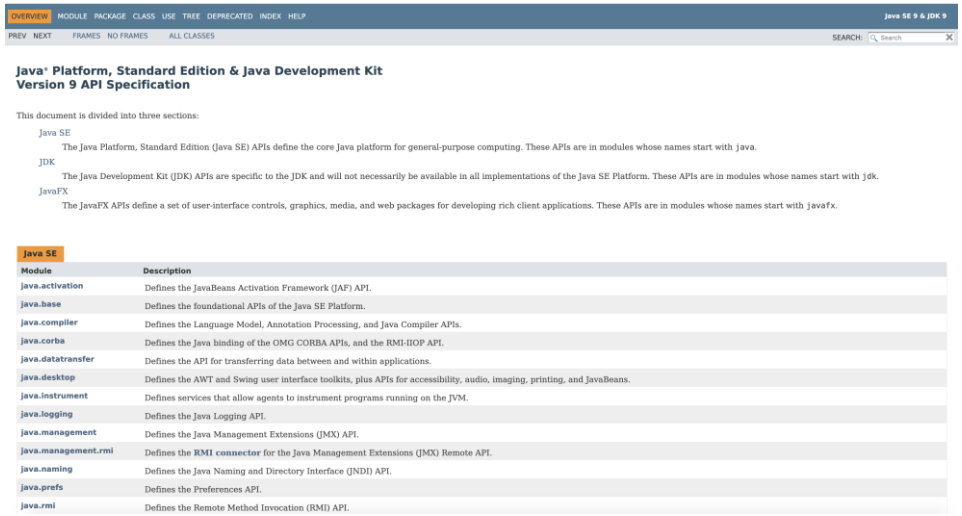
La classe String

Un oggetto String è immutabile, quindi, una volta istanziata una stringa il suo contenuto non può cambiare.

La classe String

Per conoscere la classe String e tutte le altre classi, basta andare a consultare la documentazione:

https://docs.oracle.com/javase/9/docs/api/overview-summary.html



OVERVIEW MODULE PACKAGE CLASS USE TREE DEPRECATED INDEX HELP Java SE 9 & JDK 9

PREV NEXT FRAMES NO FRAMES ALL CLASSES SEARCH: Search

Java® Platform, Standard Edition & Java Development Kit
Version 9 API Specification

This document is divided into three sections:

Java SE
The Java Platform, Standard Edition (Java SE) APIs define the core Java platform for general-purpose computing. These APIs are in modules whose names start with `java`.

JDK
The Java Development Kit (JDK) APIs are specific to the JDK and will not necessarily be available in all implementations of the Java SE Platform. These APIs are in modules whose names start with `jdk`.

JavaFX
The JavaFX APIs define a set of user-interface controls, graphics, media, and web packages for developing rich client applications. These APIs are in modules whose names start with `javafx`.

Java SE

Module	Description
java.activation	Defines the JavaBeans Activation Framework (JAF) API.
java.base	Defines the foundational APIs of the Java SE Platform.
java.compiler	Defines the Language Model, Annotation Processing, and Java Compiler APIs.
java.corba	Defines the Java binding of the OMG CORBA APIs, and the RMI-IIOP API.
java.datatransfer	Defines the API for transferring data between and within applications.
java.desktop	Defines the AWT and Swing user interface toolkits, plus APIs for accessibility, audio, imaging, printing, and JavaBeans.
java.instrument	Defines services that allow agents to instrument programs running on the JVM.
java.logging	Defines the Java Logging API.
java.management	Defines the Java Management Extensions (JMX) API.
java.management.rmi	Defines the **RMI connector** for the Java Management Extensions (JMX) Remote API.
java.naming	Defines the Java Naming and Directory Interface (JNDI) API.
java.prefs	Defines the Preferences API.
java.rmi	Defines the Remote Method Invocation (RMI) API.

Lo strumento javadoc

Lo strumento **javadoc** permette di generare ipertesti come quello della libreria standard con le informazioni sulle classi.

Lo strumento javadoc

Bisogna solamente:

1. Scrivere all’interno del commento il codice accompagnato dai commenti da formattare. Bisogna utilizzare commenti del terzo tipo, quelli compresi tra /** e */.

Lo strumento javadoc

2. Utilizzare il tool javadoc. Dal prompt, digitare:

javadoc nomeFile.java

e saranno generati automaticamente tutti i file HTML che servono.

Domande e approfondimenti